

技术报告 · 性能测试 · 第四次

PiLore 上下文缓存 利用率测试报告

多 Pack 架构下的 512K 真实对话压测：Code Pack 动态加载工具
集，追加式 Profile 上下文，累计命中率 97.82%

PiLore Team

V2.0 · 2026.08

基于 DeepSeek API 实测数据

| 目录

01	执行摘要	03
02	背景与问题定义	04
03	方法与发现	05
04	结论与建议	08
05	附录	09

| 执行摘要

PiLore 重构为多 Pack 架构(领域无关 Core + 动态加载的 DomainPack)后, 本次仅启用 Code Pack 重跑 512K 真实对话测试: 累计缓存命中率 **97.82%**, 末轮上下文 **522,741 tokens**, 估算成本 **\$0.145**, 动态工具集仅引入一次性的前缀成本, 此后保持稳定。

核心 Takeaways

- 命中率: 累计 **97.82%**, 切换轮 97.83% 与稳定轮 97.82% 差距小于 0.01 个百分点——Profile 切换零衰减。
- 动态工具集: workspace / execution 两个工具组在第 1 轮由 `activate_toolset` 一次加载, 此后 100 次调用的 tools 定义不再变化, 缓存前缀全程稳定。
- 真实负载: 54 轮对话、101 次 LLM 调用、**40 次工具调用**(`write_file` 17 · `run_code` 20 · `read_file` 3)、18 次 Profile 主动切换, 全程 0 次 HTTP 重试、0 次失败回滚。

本文档回答的问题

1. 多 Pack 架构下, 动态加载工具集(`activate_toolset`)会不会反复破坏前缀缓存?
2. Code Pack 的真实教学负载(工具链 + Profile 切换)在 512K 上下文下能维持多高的命中率?
3. 重构后的数据与追加式架构的前三次测试是否可比、可复现?

背景与问题定义

PiLore 在前三次测试后完成了一次架构重构：从单一教育会话(createEduSession)演进为领域无关的 Core 运行时 + 可组合的 DomainPack。本次测试验证重构没有破坏已经确立的缓存行为，并回答动态工具包带来的新问题。

多 Pack 架构的三个关键变化

- Core 与领域分离：运行时(createRuntime / createSession)不再绑定任何领域；Code Pack 以 DomainPack 形式注入 Profile 集合、工具清单与快照扩展。
- 工具按需动态加载：Code Pack 的工具不再常驻，而是分成 workspace(读写工作区)与 execution(沙箱运行)两个工具组，模型首次需要时调用 activate_toolset 加载，加载后在整个会话中保持激活。
- Profile 上下文延续追加式设计：教学 Profile(Oris / Feynman / Socrates)的方法论 仍以追加式内部消息进入历史，systemPrompt 全程固定——这是前三次测试确立的缓存友好模式。

本次的核心疑问

前缀缓存对请求的每一个组成部分敏感：systemPrompt、tools 定义、消息历史。追加式 Profile 上下文已经解决了「角色切换破坏前缀」的问题，但动态工具集引入了新的变量—— activate_toolset 会改变请求携带的 tools 定义。如果工具组在会话中途反复加载、卸载，每次变化都会使前缀失效。本次测试要验证的是：真实教学负载下，这个变化究竟发生几次、代价多大。

工具集变化与角色切换的缓存代价性质相同：都只发生在「定义改变的那一次请求」。只要加载是一次性的、此后保持稳定，动态工具包对前缀缓存的影响就是一次性的。

衡量口径(与前三次一致)

维度	口径
累计缓存命中率	累计 cacheRead ÷ 累计(input + cacheRead + cacheWrite)，全部 LLM 调用求和
末轮上下文	最后一个 LLM 回合的 input + cacheRead + cacheWrite
轮归属	telemetry 按 logicalRequestId / callIndex 归属，不受事件到达顺序影响
切换轮 / 稳定轮	本轮是否发生 Profile 切换(脚本主动或模型自然 adopt)

| 方法与发现

测试驱动 Code Pack 会话入口 `createCodeMentorSession`，完整启用真实行为：动态加载工具集、调用工具、中途切换 Profile、逐轮累积上下文至 512K，并以请求级 telemetry 记录每次逻辑调用的命中数据。

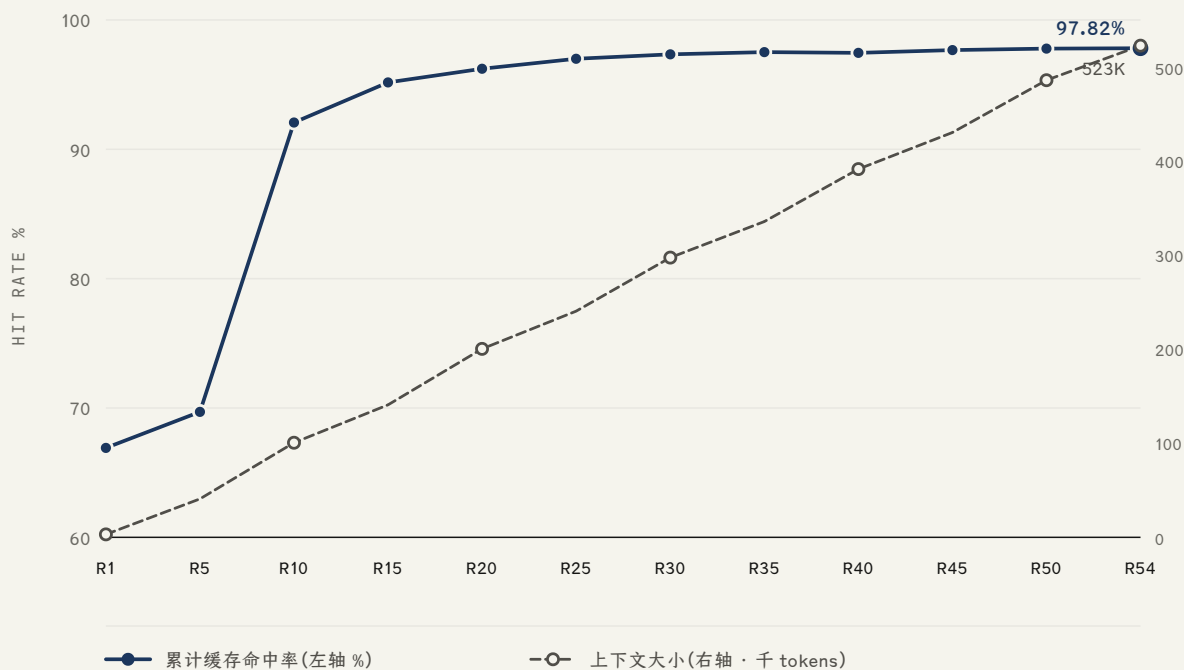
研究方法

每轮按 code → 学习材料 → 概念问答 → 学习材料 → 修 bug → 学习材料 → 问答 → 学习材料的循环模拟真实用法；每 3 轮按 Oris → Feynman → Socrates → 自动路由 的节奏切换 Profile；每轮注入约 20K tokens 的确定性学习材料推动上下文增长。执行后端为真实沙箱
192.168.172.134:1313；单轮工具链护栏 `maxTurns = 6`；达到 512K 目标即停。测试脚本同时具备失败轮回滚与 `--resume` 快照续跑能力(本次全程未触发)。

```
session = createCodeMentorSession({
  models, providerId: "deepseek", modelId: "deepseek-v4-flash",
  thinkingLevel: "off",
  profiles,                                # Oris / Feynman / Socrates(code pack)
  llmTelemetry: { onEvent },               # 请求级归属:logicalRequestId / callIndex
  maxTurns: 6,                             # 工具链护栏
})
for r in range(rounds):
  session.setProfile(rotation(r % 3))      # 中途切换 Profile
  await session.prompt(buildUserMessage(r), onEvent)
  # 第 1 轮模型自动 activate_toolset → workspace + execution
```

本次运行总览

指标	数值	指标	数值
最终上下文	522,741 tokens	对话轮数	54 轮
LLM 逻辑调用	101 次	HTTP 请求 / 重试	101 次 / 0
累计 cacheRead	22,270,976	累计未命中 input	496,073
工具调用	40 次	Profile 切换(主动)	18 次
累计缓存命中率	97.82%	估算成本	\$0.145



命中率在 R10 越过 92% 后贴顶缓行；上下文线性增长至 523K 期间，命中率没有随规模扩大而衰减。

关键发现

发现 1: 动态工具集的缓存代价是一次性的

第 1 轮模型调用 `activate_toolset` 加载 `workspace` 与 `execution` 两个工具组(该轮 6 次调用、命中率 66.9% 为全程唯一低点)；此后 100 次调用中 `tools` 定义再未变化，工具集激活计数 `workspace` 1 次、`execution` 1 次。R10 起累计命中率站上 92%，R25 起稳定在 97% 以上。换言之，「按需加载」没有变成「反复加载」——这是本次架构对缓存最友好的地方。

发现 2: Profile 切换依旧零衰减

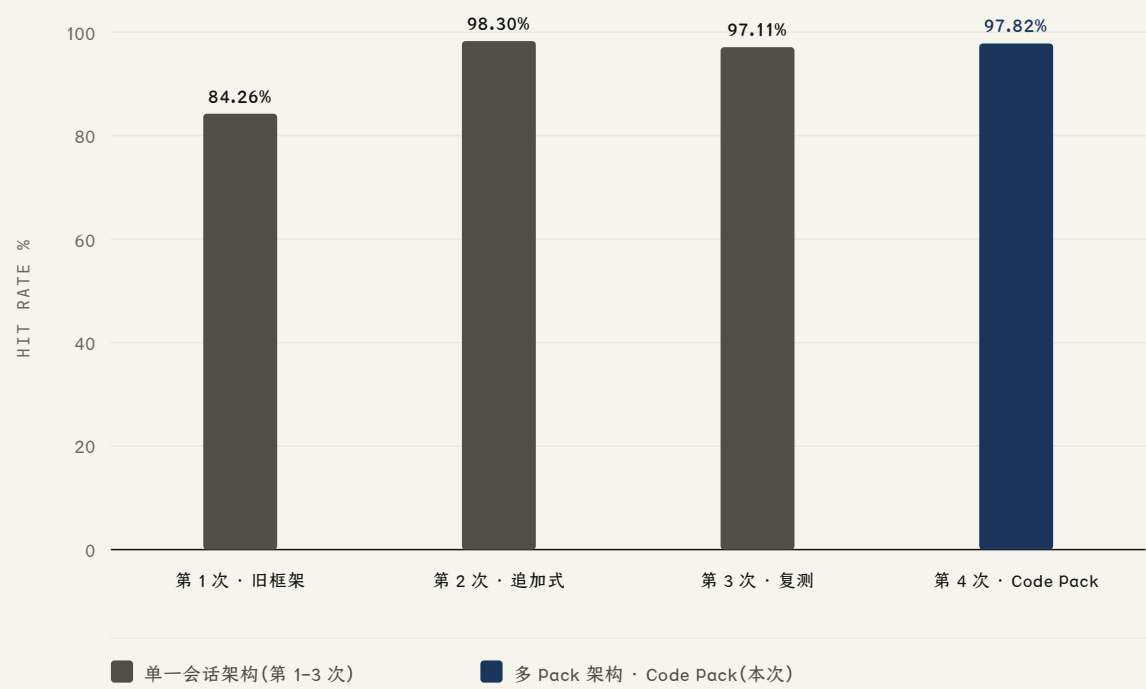
18 次脚本主动切换(Oris / Feynman / Socrates / 自动路由循环)之下，切换轮命中率 97.83%、稳定轮 97.82%，差距小于 0.01 个百分点；末轮 `commonPrefixMessages` = 202，即最后一次请求与上次共享 202 条完整历史消息。追加式 Profile 上下文在多 Pack 架构中被完整保留。

发现 3: 未命中部分约等于新增内容本身

全程最大单轮未命中 input 为 18,288 tokens，与一轮注入的学习材料(约 20K tokens)同量级；累计未命中 496,073 tokens 主要由 54 轮新增材料、模型输出回填与第 1 轮冷启动构成。缓存失效没有出现在任何一次切换或工具调用上。

发现 4：四次测试横向对比

轮次	框架	对话轮数	末轮上下文	累计命中率	未命中 input	估算成本
第 1 次	旧框架(换入式 systemPrompt)	56	526,147	84.26%	3,681,581	\$0.58
第 2 次	追加式 persona 上下文	56	549,194	98.30%	519,607	\$0.17
第 3 次	追加式 · 换新 key 复测	55	512,028	97.11%	486,880	\$0.12
第 4 次	多 Pack 架构 · Code Pack	54	522,741	97.82%	496,073	\$0.145



重构没有带来命中率回归：多 Pack 架构(97.82%)与追加式架构持平，显著高于旧框架(84.26%)。

发现 5：成本差异来自输出量，而非缓存

本次估算成本 \$0.145 略高于第 3 次的 \$0.12，两者缓存命中率相近(97.82% vs 97.11%)。差异主要来自输出 tokens：本次 47,138 vs 第 3 次 33,939——本次工具链更活跃(40 次工具调用 vs 17 次)，代码讲解更详尽。cacheRead 计费占比极低的结构没有变化。

动态加载不等于动态变化。工具组一次加载、全程保持，前缀缓存看到的就是一个稳定前缀——这与「追加式 Profile 上下文」是同一条设计原则：把变化收敛为一次性事件。

— 本次测试的核心观察

| 结论与建议

多 Pack 重构通过了 512K 真实负载的缓存考验：命中率 97.82% 与追加式架构持平，动态工具集只产生一次性前缀成本，Profile 切换依旧零衰减。

核心结论

1. 多 Pack 架构下累计缓存命中率 **97.82%**(末轮上下文 522,741 tokens)，与追加式架构的第 2、3 次测试(98.30% / 97.11%)持平，比重构前(84.26%)高 13.6 个百分点。
2. 动态工具集是缓存友好的：workspace / execution 在第 1 轮一次加载后，后续 100 次调用 tools 定义不变；切换轮与稳定轮命中率差距小于 0.01 个百分点。
3. 数据可复现、可审计：101 次逻辑调用 = 101 次 HTTP = 0 重试，telemetry 按 logicalRequestId / callIndex 逐请求归属，末轮前缀共享 202 条历史消息。

下一步建议

- 保持「systemPrompt 固定 + Profile 方法论追加 + 工具集一次加载」的架构不变。
- 若未来引入更多工具组(如 english pack 的 evaluation)，应在会话早期一次性加载完毕，避免会话中段再变更 tools 定义——那是唯一会重新付出前缀代价的操作。
- 工具链回合数是下一个优化点：本次 101 次调用中每个工具回合都重发完整历史，压缩单次任务的工具往返次数可进一步降低成本。
- 以 DeepSeek 控制台的缓存命中率与账单作为服务端侧的最终核对依据。

CALL TO ACTION

如果只需记住一件事：缓存友好不是「没有变化」，而是「变化只发生一次」。角色方法论追加进消息流、工具组首次使用时加载，512K 真实对话就能稳定维持 97% 以上的命中率。

| 附录

A. 数据来源

- reports/cache-realistic-2026-08-12T07-59-30-457Z.json(第 4 次 · 多 Pack 架构 · Code Pack, 本次报告)
- reports/cache-realistic-2026-08-10T14-31-22-668Z.json(第 1 次 · 旧框架)
- reports/cache-realistic-2026-08-10T15-41-28-797Z.json(第 2 次 · 追加式)
- reports/cache-realistic-2026-08-11T03-04-46-274Z.json(第 3 次 · 追加式 · 复测)

B. 术语表

前缀缓存(prefix caching): DeepSeek 服务端对请求前缀的自动缓存; 命中的 token 按 cacheRead 超低价计费。

DomainPack: 领域能力包, 向 Core 运行时注入 Profile 集合、工具清单与快照扩展; code pack 为编程教学领域实现。

activate_toolset: 内部工具, 按需加载 DomainPack 声明的工具组(如 workspace / execution), 加载后持久激活。

追加式 Profile 上下文: Profile 方法论作为内部消息与下一条用户消息合并, systemPrompt 全程固定。

logicalRequestId / callIndex: telemetry 中逻辑调用的唯一标识与全局递增序号, 用于请求级归属。

commonPrefixMessages: 当前请求与上次请求共享的完整历史消息条数。

C. 测试条件

模型: deepseek-v4-flash · thinking off; **执行后端**: 192.168.172.134:1313(真实沙箱); **上下文目标**: ~512K tokens; **注入节奏**: 每轮约 20K tokens 确定性学习材料; **单轮护栏**: maxTurns=6; **Profile 节奏**: 每 3 轮切换一次(Oris → Feynman → Socrates → 自动路由); **容错**: 失败轮回滚 + --resume 快照续跑(本次全程未触发)。